

```

#include <stdio.h>

#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

// Function to display linked list
void display(struct Node* head) {

```

```

struct Node* temp = head;
while (temp != NULL) {
    printf("%d -> ", temp->data);
    temp = temp->next;
}
printf("NULL\n");
}

// Function to sort linked list (Bubble Sort)
void sortList(struct Node* head) {
    if (head == NULL) return;
    struct Node* i, *j;
    int temp;
    for (i = head; i->next != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                temp = i->data;
                i->data = j->data;
                j->data = temp;}
        }
    }
}

// Function to reverse linked list
struct Node* reverseList(struct Node* head) {
    struct Node* prev = NULL;
    struct Node* curr = head;
    struct Node* next = NULL;
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;

```

```

    prev = curr;

    curr = next;

}

return prev; // new head
}

// Function to concatenate two linked lists
struct Node* concatenate(struct Node* list1, struct Node* list2) {
    if (list1 == NULL) return list2;

    struct Node* temp = list1;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = list2;
    return list1;
}

// Main function
int main() {
    struct Node* list1 = NULL;
    struct Node* list2 = NULL;

    // Insert elements into list1
    insertEnd(&list1, 30);
    insertEnd(&list1, 10);
    insertEnd(&list1, 20);

    printf("List1: ");
    display(list1);

    // Sort list1
    sortList(list1);

    printf("Sorted List1: ");
    display(list1);
}

```

```

// Reverse list1
list1 = reverseList(list1);
printf("Reversed List1: ");
display(list1);
// Insert elements into list2
insertEnd(&list2, 40);
insertEnd(&list2, 50);
printf("List2: ");
display(list2);
// Concatenate list1 and list2
struct Node* mergedList = concatenate(list1, list2);
printf("Concatenated List: ");
display(mergedList);

return 0;
}

```

OUTPUT:

```

List1: 30 -> 10 -> 20 -> NULL
Sorted List1: 10 -> 20 -> 30 -> NULL
Reversed List1: 30 -> 20 -> 10 -> NULL
List2: 40 -> 50 -> NULL
Concatenated List: 30 -> 20 -> 10 -> 40 -> 50 -> NULL

Process returned 0 (0x0)    execution time : 0.247 s
Press any key to continue.

```